

شاملة للمشروع LinkDev — Documentation

وصف مختصر: LinkDev هو نظام متكامل لإدارة البطولات والمنافسات الرياضية، مبني باستخدام Laravel، ويغطي دورة البطولة من تسجيل الفرق وقبولها، مرورًا بتكوين المجموعات وتوليد المباريات، ووصولًا إلى المتابعة اللحظية والترتيب والأدوار الإقصائية وتحديد البطل.

1. نطاق الوثيقة

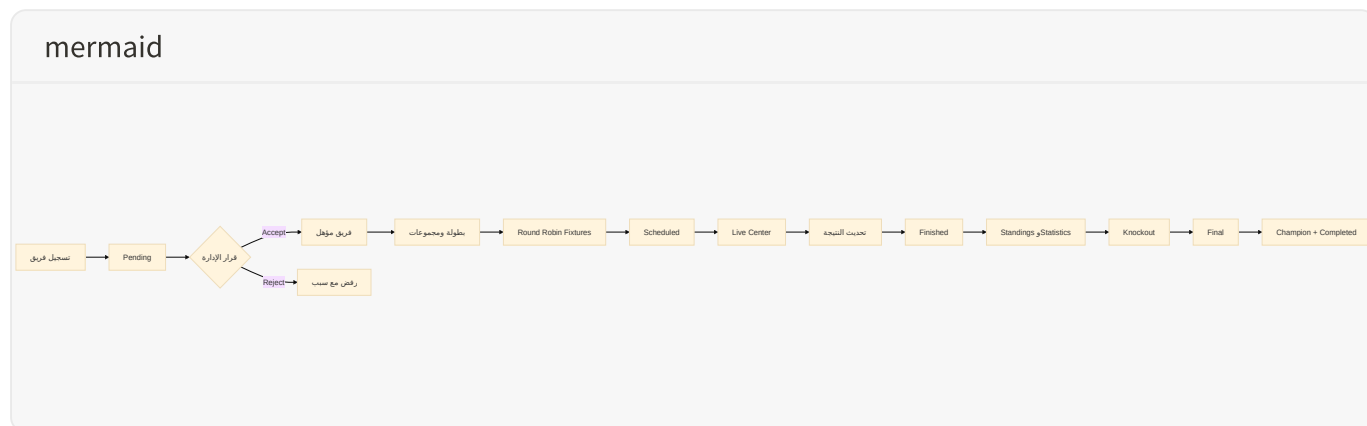
تشرح هذه الوثيقة البنية الحالية للمشروع في فرع `develop`، كما توضح دورة الـ `Business Logic`، خوارزميات توليد المباريات، إدارة التواريخ، آلية تحديث الترتيب، وآلية استخدام `Transactions` و `Events` و `Jobs` و `Queues`. الشرح مبني على مراجعة الكود والملفات الموجودة في المستودع، وليس على اقتراحات خارجية.

الإصدار محل التوثيق هو `659ba95` في فرع `develop`.

2. نظرة عامة على النظام

ليس مجرد شاشة لعرض نتائج المباريات؛ بل يعمل كمحرك لإدارة البطولة. النظام يفصل بين LinkDev الواجهة العامة، ولوحة الإدارة، ومنطق الأعمال، وتخزين البيانات، والتحديثات اللحظية.

يمكن تلخيص دورة النظام كالتالي:



3. المكونات الرئيسية

المكوّن	المسؤولية
Routes	تعريف الواجهات العامة والإدارية ومسارات الطلبات والقنوات
Controllers	استقبال الطلبات وتمريرها إلى طبقة الخدمات
Form Requests	التحقق من المدخلات والصلاحيات والحقول المسموحة

Services	تنفيذ منطق البطولة والمباريات والفرق والمستخدمين
Repositories	التعامل مع الاستعلامات وحفظ البيانات
Models	تمثيل الكيانات وعلاقات Eloquent
Jobs	تنفيذ العمليات القابلة للتأجيل مثل البريد والبريد
Events	إعلان وقوع حدث، خصوصًا للتحديث اللحظي
Queue	تخزين الـ Jobs قبل تنفيذها بواسطة Worker
Views/JavaScript	عرض لوحات الإدارة والواجهة العامة و Live Center
Tests	تغطية التسجيل والترتيب و Knockout وبعض منطق الأعمال

4. أهم الـ Features

4.1 الواجهة العامة

تعرض الواجهة العامة البطولات والفرق والمباريات، وتفصل بين المباريات القادمة والمباريات المباشرة. يستطيع المستخدم فتح تفاصيل البطولة، مشاهدة الفرق، متابعة الترتيب، والاطلاع على Bracket الأدوار الإقصائية.

4.2 الحسابات والأمان

يدعم النظام إنشاء الحساب وتسجيل الدخول والخروج وتأكد البريد الإلكتروني باستخدام OTP واستعادة كلمة المرور. يوجد ملف شخصي منفصل للمستخدم، وتعديل الملف لا يلغي حالة التحقق من البريد.

4.3 تسجيل الفرق ومراجعتها

تسجيل الفريق يبدأ بحالة Pending. بعد ذلك تستطيع الإدارة قبول الفريق أو رفضه، مع إمكانية حفظ سبب الرفض وإرسال رسالة بريدية. الفريق المقبول فقط يمكن إضافته إلى مجموعة. كما تسجل العمليات الإدارية الحساسة في ActivityLog.

4.4 إدارة البطولات

يمكن إنشاء وتعديل وحذف البطولة، مع ضبط الموسم والتواريخ والحالة والفائز وإعدادات المنافسة. حالات البطولة الرئيسية هي:

Plain Text

draft → upcoming → ongoing → completed

وتوجد حالة cancelled التي يجب ألا تتغير تلقائيًا أثناء مزامنة الحالة.

4.5 إدارة المجموعات

يمكن إنشاء مجموعات داخل البطولة وربط الفرق بها. يفرض النظام قاعدتين مهمتين: الفريق غير المقبول لا يدخل المجموعة، والفريق لا يمكن أن يتكرر في مجموعتين داخل البطولة نفسها.

4.6 إدارة المباريات

تدعم الخدمة إنشاء المباراة وتعديلها وحذفها وتوليد مباريات المجموعة تلقائيًا وفتح Live Center وتحديث النتيجة وإنهاء المباراة.

4.7 Live Center

يسمح Live Center ببدء المباراة، زيادة أو تقليل نتيجة الفريق المضيف أو الضيف، ثم إنهاء المباراة. عند كل تغيير مهم يمكن إرسال تحديث لحظي إلى الشاشات العامة ولوحة الإدارة.

4.8 Standings وStatistics

يعيد النظام حساب ترتيب المجموعة وإحصائيات الفرق بناءً على نتائج المباريات. إعادة الحساب مصممة لتكون Idempotent ؛ أي أن تشغيلها عدة مرات لا يضاعف النقاط أو عدد المباريات.

4.9 Knockout

بعد اكتمال دور المجموعات، ينشئ النظام مباريات الأدوار الإقصائية حسب عدد المجموعات. بعد اكتمال كل جولة يتأهل الفائزون للجولة التالية، وعند انتهاء النهائي يتم تعيين بطل البطولة.

5. بنية قاعدة البيانات

يحتوي المشروع على 12 جدولًا رئيسيًا:

الجدول	الغرض
users	المستخدمون الأساسيون
staff_profiles	بيانات طاقم العمل والإدارة
teams	الفرق والشعارات وبيانات النادي
competitions	البطولات وحالاتها وتواريخها والفائز
competition_settings	نوع البطولة ونظام النقاط
competition_team	ربط الفرق بالبطولات
competition_groups	مجموعات البطولة

group_team	ربط الفرق بالمجموعات
matches	المباريات والنتائج والحالات والتواريخ
group_standings	ترتيب الفرق داخل المجموعة
team_statistics	إحصائيات الفريق على مستوى البطولة
activity_logs	سجل العمليات الإدارية

5.1 جدول competitions

يحتوي على:

Plain Text

```
name
season
status
start_date
end_date
winner_team_id
created_by_user_id
```

5.2 جدول matches

يحتوي على:

Plain Text

```
competition_id
group_id
home_team_id
away_team_id
winner_team_id
scheduled_at
started_at
ended_at
status
home_score
away_score
round_number
```

معنى الحقول الزمنية:

الحقل	المعنى
scheduled_at	الموعد المخطط للمباراة
started_at	وقت بدء المباراة فعليًا
ended_at	وقت نهاية المباراة فعليًا

6. طبقة الـ Services

6.1 MatchService

هي طبقة التنسيق الرئيسية لتدفقات المباريات. من أهم دوالها:

الدالة	الوظيفة
createMatch	إنشاء مباراة جديدة وإرسال إشعار إذا بدأت Live
generateFixtures	حذف الجدول غير النشط وإعادة توليد Fixtures داخل Transaction
updateMatch	تحديث المباراة وإعادة حساب الترتيب والإحصائيات
destroyMatch	حذف المباراة ومزامنة البيانات التابعة
startMatchLive	بدء المباراة وتسجيل وقت البداية
incrementScore	زيادة نتيجة فريق
decrementScore	تقليل نتيجة فريق مع منع القيم غير الصحيحة
finishMatch	إنهاء المباراة وتحديد الفائز والتحقق من التعادل في Knockout

6.2 FixtureGeneratorService

مسؤولة عن توليد مباريات دور المجموعات باستخدام Circle Method. إذا كان عدد الفرق فرديًا، يضيف النظام قيمة `null` لتمثيل `Bye`، ولا ينشئ مباراة حقيقية ضدها.

6.3 MatchStandingsService

مسؤولة عن إعادة حساب ترتيب المجموعات، إحصائيات الفرق، إنشاء Knockout، دفع الفائزين للجولة التالية، ومزامنة تاريخ وحالة البطولة.

6.4 MatchLiveStatusService

تبحث عن المباريات التي حالتها `scheduled` وموعدها وصل، ثم تحولها إلى `live` ، وتسجل `started_at` ، وتحديث حالة البطولة إلى `ongoing` ، ثم ترسل Jobs للبث والإشعار.

7. خوارزمية Round Robin

إذا كان عدد الفرق `N` زوجيًا، فإن:

Plain Text

عدد الجولات = $N - 1$
عدد المباريات في الجولة = $N / 2$
إجمالي المباريات = $N \times (N - 1) / 2$

الخوارزمية تثبت أول فريق وتدور بقية الفرق بعد كل جولة. يتم إنشاء المباراة بهذه القاعدة:

PHP

```
$home = $teamIds[$i];  
$away = $teamIds[$numTeams - 1 - $i];
```

بعد ذلك يتم تدوير القائمة:

PHP

```
$last = array_pop($teamIds);  
array_splice($teamIds, 1, 0, [$last]);
```

7.1 الفاصل الزمني داخل الجولة

المباريات داخل الجولة نفسها تفصلها ساعتان:

PHP

```
'scheduled_at' => $matchDate->copy()->addHours($i * 2)
```

مثلاً:

Plain Text

17:00
19:00
21:00

7.2 الفاصل بين الجولات

الفاصل الافتراضي بين الجولات يومان:

PHP

```
$daysBetweenRounds = 2;
```

إذا كانت البطولة تحتوي على `end_date` ، يحاول الكود توزيع الجولات بين تاريخ البداية والنهاية. لذلك قد يصبح الفاصل 3 أو 4 أيام بدل يومين.

8. خوارزمية Knockout

لا يبدأ توليد Knockout إلا بعد انتهاء كل مباريات المجموعات:

PHP

```
$unfinishedGroupMatches = $groupMatches  
->where('status', '!=', 'finished');  
  
if ($unfinishedGroupMatches->isNotEmpty()) {  
    return;  
}
```

كما يمنع الكود إنشاء Knockout أكثر من مرة في الطلبات المتسلسلة:

PHP

```
$existingKnockoutMatches = $competition->matches()  
->whereNull('group_id')  
->exists();
```

8.1 مجموعتان

Plain Text

متصدر المجموعة الأولى × وصيف المجموعة الثانية
متصدر المجموعة الثانية × وصيف المجموعة الأولى

8.2 أربع مجموعات

Plain Text

1A × 2B
1C × 2D
1B × 2A
1D × 2C

8.3 الجولة التالية

بعد انتهاء كل مباريات الجولة الحالية، يجمع النظام الفائزين وينشئ الجولة التالية بعد 3 أيام من آخر موعد
مجدول في الجولة الحالية:

PHP

```
$nextRoundDate = Carbon::parse($lastMatchDate)
    ->addDays(3)
    ->setHour(17)
    ->setMinute(0);
```

8.4 التعادل في Knockout

في المباراة العادية، التعادل مسموح ويظل `winner_team_id` فارغًا. أما Knockout، فلا يسمح النظام بإنهاء
المباراة بالتعادل دون تحديد فائز:

PHP

```
if ($isKnockout && $homeScore === $awayScore && ! $winnerId) {
    throw new DomainException(
        'Knockout match requires a designated winner.'
    );
}
```

9. إدارة تواريخ البطولة والمباريات

9.1 بداية المباريات

أولوية تاريخ البداية هي:

Plain Text

1. المرسل يدويًا startDate
2. الساعة 17:00 competition.start_date
3. غدًا الساعة 17:00

9.2 مزامنة تواريخ البطولة

الدالة:

PHP

```
syncCompetitionDatesAndStatus($competition)
```

تقرأ كل مباريات البطولة ثم تحسب:

PHP

```
$minDate = $matches->min('scheduled_at');  
$maxDate = $matches->max('scheduled_at');
```

ثم تحفظ:

PHP

```
competition.start_date = min(all matches.scheduled_at)  
competition.end_date   = max(all matches.scheduled_at)
```

لذلك `competition.end_date` هو تاريخ أبعد مباراة مجدولة، وليس `ended_at` الفعلي للنهائي. إذا كان النهائي هو آخر مباراة مجدولة، فسيكون تاريخ النهائي مساويًا لتاريخ نهاية البطولة. لكن لا توجد علاقة مباشرة تقول إن نهاية البطولة يجب أن تساوي النهائي؛ القاعدة الحالية تعتمد على `MAX(scheduled_at)`.

9.3 حالات البطولة

الشرط	الحالة
كل المباريات Finished	completed
توجد مباراة Live أو Finished فقط	ongoing
المواعيد الحالية داخل نطاق البطولة	ongoing
لم تبدأ المباريات بعد	upcoming
البطولة cancelled	لا يتم تغييرها تلقائيًا

10. Transactions

10.1 ما هي Transaction؟

Transaction تجمع عدة عمليات قاعدة بيانات في وحدة واحدة. النتيجة المطلوبة هي:

Plain Text

إما أن تنجح كل العمليات
لكل التغييرات Rollback أو يتم

مثال عام:

PHP

```
DB::transaction(function () {  
    // تحديث المباراة  
    // إعادة حساب الترتيب  
    // تحديث إحصائيات الفرق  
    // مزامنة البطولة  
});
```

10.2 أين يستخدمها المشروع؟

يستخدم المشروع DB::transaction() في عمليات مهمة، منها:

العملية	ما الذي تحميه Transaction؟
إنشاء مباراة	حفظ المباراة والبيانات التابعة بشكل متسق
توليد Fixtures	حذف الجدول القديم وإنشاء الجدول الجديد كوحدة واحدة
تحديث مباراة	تحديث النتيجة وإعادة حساب الترتيب والإحصائيات
حذف مباراة	حذف المباراة ومزامنة الترتيب والإحصائيات
تحديث Live Score	منع حفظ جزء من عملية النتيجة دون بقية التغييرات
إعادة حساب Standings	حفظ ترتيب المجموعة وإحصائيات الفريق معًا
إنشاء المستخدم والإدارة	حفظ المستخدم والملف والدور بدون حالة جزئية
إدارة الأدوار	تحديث الدور والصلاحيات في عملية متسقة

10.3 مثال تحديث مباراة

PHP

```

$updatedMatch = DB::transaction(function () use ($match, $updatePayload) {
    $updated = $this->matchRepository->update(
        $match,
        $updatePayload
    );

    if ($updated->group) {
        $this->standingsService
            ->recalculateGroupStandings($updated->group);
    }

    if ($updated->competition) {
        $this->standingsService
            ->recalculateTeamStatistics($updated->competition);

        if ($updated->status === 'finished') {
            $this->standingsService
                ->checkAndGenerateKnockoutFromGroups(
                    $updated->competition
                );

            $this->standingsService
                ->advanceKnockoutWinner($updated);
        }

        $this->standingsService
            ->syncCompetitionDatesAndStatus(
                $updated->competition
            );
    }

    return $updated;
});

```

10.4 الميزة في LinkDev

بدون Transaction قد يحدث السيناريو التالي:

Plain Text

1. حفظ نتيجة المباراة
2. فشل تحديث الترتيب
3. بقاء النتيجة الجديدة مع ترتيب قديم

أما مع Transaction، إذا فشلت خطوة إعادة الحساب، يتم Rollback للنتيجة أيضًا، فلا يصبح النظام في حالة متناقضة.

10.5 حدود Transaction الحالية

وجود Transaction داخل MatchService لا يضمن تلقائيًا أن كل سلسلة كبيرة أصبحت ذرية بالكامل. يجب مراجعة حدود العملية عند إنهاء المباراة، خصوصًا إذا كان تدفق واحد يشمل:

Plain Text

Update Match

- Recalculate Standings
- Generate Knockout
- Advance Winner
- Update Competition

كما أن إعداد Queue database الحالي يحتوي على:

PHP

```
'after_commit' => false
```

لذلك يجب الحذر من إرسال Job قبل تأكيد Commit في بعض التدفقات. الحل الأفضل في العمليات الحساسة هو استخدام `afterCommit()` عند Dispatch أو ضبط الإعداد بما يتوافق مع سياسة المشروع.

11. Events

11.1 معنى Event

الـ Event يمثل حدثًا وقع داخل النظام:

Plain Text

المباراة بدأت
النتيجة تغيرت
المباراة انتهت
تم التحقق من البريد

الـ Event لا يجب أن يحتوي على كل منطق التنفيذ؛ دوره الأساسي إعلان الحدث وإرسال البيانات المطلوبة للمستمعين أو البث.

11.2 Events الخاصة بالمباريات

Event	القنوات	الاستخدام
MatchStartedLiveEvent	live-matches , matches	إعلان بداية المباراة

MatchStatusUpdatedEvent	live-matches , match.{id}	تحديث الحالة والنتيجة والفائز
AdminLiveMatchNotificationEvent	admin-notifications	إشعار الإدارة ببدء مباراة

11.3 ShouldBroadcastNow

الأحداث اللحظية في المشروع تستخدم:

PHP

```
implements ShouldBroadcastNow
```

وهذا يعني أن البث يتم فور استدعاء الـ Event من داخل الـ Job، بدل وضع عملية البث نفسها في Queue إضافية.

مثال:

PHP

```
event(new MatchStartedLiveEvent($this->match));
```

ثم يحدد الـ Event القنوات:

PHP

```
public function broadcastOn(): array
{
    return [
        new Channel('live-matches'),
        new Channel('matches'),
    ];
}
```

ويحدد اسم الحدث:

PHP

```
public function broadcastAs(): string
{
    return 'MatchStartedLive';
}
```

ويحدد الـ Payload:

PHP

```
public function broadcastWith(): array
{
    return [
        'match' => $this->matchData,
    ];
}
```

11.4 وصول Event للواجهة

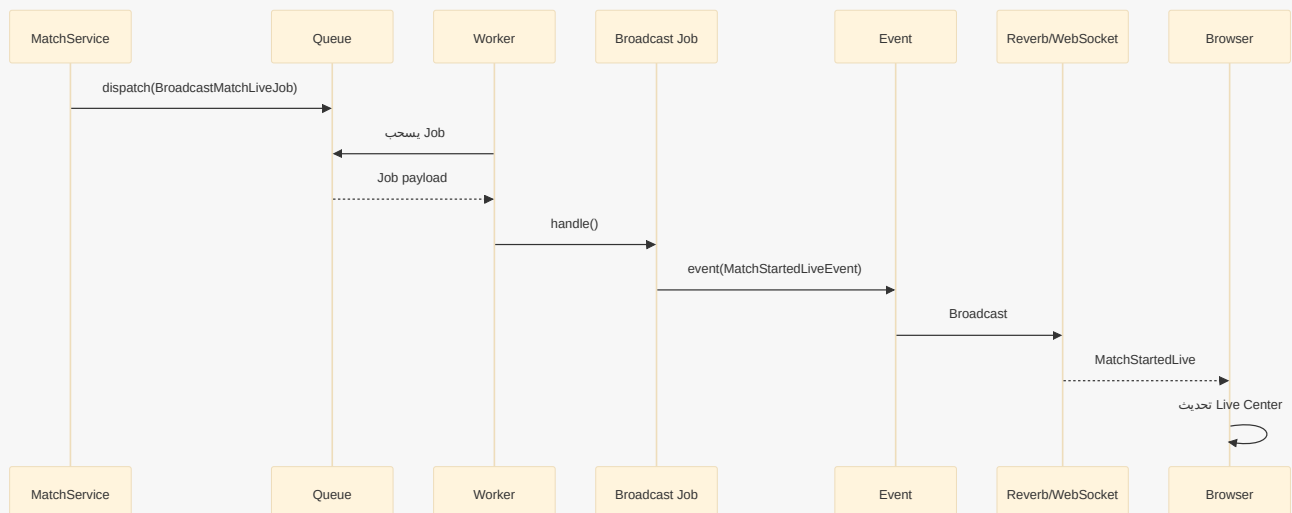
تستخدم الواجهة Laravel Echo للاستماع:

JavaScript

```
window.Echo.channel('live-matches')
    .listen('.MatchStartedLive', function (data) {
        // إضافة أو تحديث كارت المباراة
    });
```

التدفق يصبح:

mermaid



12. Jobs

12.1 معنى Job

الJob هو وحدة عمل مستقلة يمكن تنفيذها لاحقًا. يستخدمه المشروع للعمليات التي لا يجب أن تعطل Request المستخدم، مثل إرسال البريد أو بث تحديث للمستخدمين.

12.2 Jobs مهمة في المشروع

Job	الوظيفة
BroadcastMatchLiveJob	إطلاق Events عند بدء المباراة
BroadcastMatchStatusJob	إطلاق Event عند تغيير حالة المباراة
BroadcastMatchScoreUpdateJob	بث تحديثات النتيجة أثناء Live
SendMatchLiveNotificationJob	إرسال بريد بدء المباراة
SendOtpNotificationJob	إرسال OTP
SendEmailVerificationOtpJob	إرسال OTP تأكيد البريد
SendTeamAcceptedEmailJob	إرسال قبول الفريق
SendTeamRejectedEmailJob	إرسال رفض الفريق وسببه

12.3 مثال Job البث

PHP

```
class BroadcastMatchLiveJob implements ShouldQueue
{
    use Dispatchable;
    use InteractsWithQueue;
    use Queueable;
    use SerializesModels;

    public int $tries = 3;
    public int $backoff = 2;

    public function __construct(
        public GameMatch $match
    ) {}

    public function handle(): void
    {
        $this->match->loadMissing([
            'competition',
            'group',
            'homeTeam',
            'awayTeam',
        ]);
    }
}
```

```

        event(new MatchStartedLiveEvent($this->match));
        event(new AdminLiveMatchNotificationEvent($this->match));
    }
}

```

12.4 Retry وBackoff

بعض Jobs تحتوي على:

PHP

```

public int $tries = 3;
public int $backoff = 2;

```

معناها أن Laravel يحاول التنفيذ ثلاث مرات، و ينتظر ثانيتين قبل إعادة المحاولة. Jobs البريد قد تستخدم عدد محاولات أو Backoff مختلفًا بسبب طبيعة SMTP والشبكة.

13. Queue

13.1 إعداد Queue الحالي

في `env.example`:

Plain Text

`QUEUE_CONNECTION=database`

وفي `config/queue.php`:

PHP

```

'database' => [
    'driver' => 'database',
    'table' => env('DB_QUEUE_TABLE', 'jobs'),
    'queue' => env('DB_QUEUE', 'default'),
]

```

بذلك عندما يتم تنفيذ:

PHP

```

BroadcastMatchLiveJob::dispatch($match);

```

يتم تخزين الـ Job في جدول `jobs` ، ثم يقوم Worker بتنفيذه.

13.2 تشغيل Worker

بعد تجهيز الجداول والبيئة:

Bash

```
php artisan queue:work
```

لمراقبة Jobs الفاشلة:

Bash

```
php artisan queue:failed
```

ولإعادة Job فاشل:

Bash

```
php artisan queue:retry all
```

13.3 الفرق بين Queue و Worker

المصطلح	المعنى
Queue	مكان تخزين Jobs المنتظرة
Job	المهمة نفسها
Worker	وينفذها Jobs يسحب Process
Failed Jobs	فشلت بعد عدد المحاولات المسموح Jobs

14. تدفق Live Match كامل

Plain Text

- المستخدم يفتح صفحة المباريات أو يستدعي مسارًا ينفذ فحص الحالة.
- MatchLiveStatusService يبحث عن `scheduled_at <= now`.
- `started_at` ويتم تسجيل live المباراة تتحول إلى.
- عند الحاجة ongoing حالة البطولة تتحول إلى.
- Dispatch لـ BroadcastMatchLiveJob.
- Dispatch لـ SendMatchLiveNotificationJob.
- Jobs يسحب ال Queue Worker.

8. BroadcastMatchLiveJob يطلق MatchStartedLiveEvent.
9. Event و matches و live-matches يتم بثه إلى Event.
10. Laravel Echo في المتصفح يستقبل الحدث.
11. JavaScript يضيف أو يحدّث بطاقة المباراة.
12. Job البريد يرسل الإشعار للمستخدمين.

15. الفرق بين Transaction و Job و Event و Queue

العنصر	سؤال يجيب عنه	مثال في LinkDev
Transaction	كيف أحافظ على اتساق قاعدة البيانات؟	تحديث المباراة + الترتيب + الإحصائيات
Event	ماذا حدث؟	المباراة بدأت Live
Job	ما العمل الذي يمكن تنفيذه منفصلاً؟	بث الحدث أو إرسال البريد
Queue	أين أنتظر تنفيذ العمل؟	جدول jobs
Worker	من ينفذ العمل؟	php artisan queue:work

16. أفضل الممارسات المقترحة

16.1 استخدام afterCommit

لمنع إرسال Job يحتوي على Model قبل Commit، يفضل استخدام:

PHP

```
BroadcastMatchStatusJob::dispatch($updatedMatch)
->afterCommit();
```

أو ضبط استراتيجية Queue بحيث لا تُنفذ Jobs إلا بعد نجاح Transaction.

16.2 منع تكرار Knockout تحت التزامن

الفحص الحالي باستخدام `exists()` يمنع التكرار في الطلبات المتسلسلة، لكنه قد لا يمنع Race Condition بين طلبين متزامنين. يفضل إضافة Lock أو Business Key فريد داخل Transaction.

16.3 فصل القنوات العامة عن الإدارية

يجب مراجعة قنوات مثل:

Plain Text

```
admin-notifications
live-matches
match.{id}
```

والتأكد أن بيانات الإدارة لا تخرج إلى قناة عامة إذا كانت تحتوي على معلومات داخلية.

16.4 إضافة Scheduler لفحص المباريات

إذا كان تحويل المباراة إلى `live` يعتمد فقط على زيارة صفحة أو Request، فقد لا يحدث التحديث في الوقت المتوقع عند عدم وجود مستخدم يفتح الصفحة. يفضل إضافة Command دوري:

Bash

```
php artisan make:command UpdateLiveMatchStatuses
```

ثم تشغيله من Scheduler كل دقيقة، بحيث ينفذ `checkAndUpdateLiveStatuses()` دون الاعتماد على حركة المستخدم.

16.5 عدم وضع عمليات ثقيلة داخل Event

الـ Event يجب أن يعلن الحدث ويرسل Payload صغيرًا. إرسال البريد أو تنفيذ عمليات ثقيلة يجب أن يكون داخل Job مستقل.

17. التشغيل المحلي المقترح

Bash

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate:fresh --seed
php artisan queue:work
```

وفي بيئة التطوير، يجب ضبط إعدادات قاعدة البيانات والبريد واللبث مثل:

Plain Text

```
QUEUE_CONNECTION=database
BROADCAST_CONNECTION=reverb
MAIL_MAILER=log
```

القيمة الموجودة في `env.example` لللبث هي `log` لأغراض التطوير، ويمكن استخدام Reverb أو Pusher في بيئة البث الفعلية حسب البنية المعتمدة.

18. الاختبارات

أهم الاختبارات الموجودة في الفرع تشمل:

الاختبار	الغرض
MatchStandingsTest	التأكد من أن إعادة حساب الترتيب عدة مرات لا تضاعف النتائج
KnockoutStageTest	اختبار إنشاء نصف النهائي والنهائي وتعيين فائز التعادل
اختبارات التسجيل	التحقق من تدفق تسجيل المستخدم

يوصى بإضافة اختبارات HTTP لتدفق Live Center تشمل:

Plain Text
بدء المباراة زيادة النتيجة تقليل النتيجة منع النتيجة السالبة متعادلة مع فائز Knockout إنهاء مباراة متعادل بلا فائز Knockout رفض إنهاء التحقق من الصلاحيات

19. المخاطر الحالية

المعالجة المقترحة	الأثر	الخطر
إضافة اختيار فائز إلزامي في الواجهة والطلب	قد يتوقف تقدم Knockout	غير مرسل من winner_team_id عند التعادل Live Center
Lock أو Unique Business Key	احتمال إنشاء جولات مكررة	عند إنشاء Race Condition Knockout
تعريف Preliminary Round أو منع العدد الفردي	غير مكتمل Bracket	عدم اكتمال سياسة العدد الفردي من المجموعات
اختبارات 4 مجموعات وجولات متعددة	عدم ربط كل الفائزين بالجولة التالية	عدم اكتمال Bracket الكبير
دوري Scheduler	تأخر الانتقال إلى Live	الاعتماد على Request لتحديث Live Status
afterCommit()	يقرأ بيانات غير نهائية Job	Dispatch قبل Commit

20. الخلاصة التنفيذية

يملك فصلًا جيدًا بين منطق الأعمال والتعامل مع البيانات والتشغيل غير المتزامن. LinkDev تنقل البريد والبث خارج دورة طلب المستخدم، Jobs تحمي اتساق العمليات الرئيسية، والـ JTransactions توفر مكانًا لإدارة التنفيذ المؤجل وإعادة المحاولة Queue تعلن تغييرات المباريات، والـ Events والـ القيمة العملية لهذا التصميم هي أن تحديث النتيجة يستطيع أن يمر في مسار منظم:

Plain Text

- تحديث المباراة
- Transaction
- Standings إعادة حساب
- Statistics تحديث
- Knockout توليد أو تقدم
- Dispatch Job
- Event Broadcast
- تحديث الواجهة

وللوصول إلى جاهزية إنتاجية أقوى، يجب التركيز على أربعة أمور: إكمال اختيار الفائز في Live Center عند التعادل، استخدام afterCommit للـ Jobs، حماية توليد Knockout من التزامن، وإضافة Scheduler فعلي لتحديث حالة المباريات تلقائيًا.

References

1. [Project README — ReadMe.md](#)
2. [MatchService](#)
3. [FixtureGeneratorService](#)
4. [MatchStandingsService](#)
5. [MatchLiveStatusService](#)
6. [BroadcastMatchLiveJob](#)
7. [BroadcastMatchStatusJob](#)
8. [MatchStartedLiveEvent](#)
9. [MatchStatusUpdatedEvent](#)
10. [AdminLiveMatchNotificationEvent](#)

11. [Queue Configuration](#)
12. [Broadcast Configuration](#)
13. [Broadcast Channels](#)
14. [Live Center View](#)
15. [Develop Business Logic Analysis](#)